Project Setup and First Run Walkthrough

Jetpack Compose Project Initialization and Execution Report

Submitted By

Priyanka K

Internship Course

Android App Development using Gen AI

Task 1 Submission

February 19, 2026

1. Environment Checklist

Android Studio Version	Android Studio Panda 1 | 2025.3.1 Patch 1
Build Number	AI-253.29346.138.2531.14876573
Runtime	OpenJDK 21 (64-bit)
Operating System	Windows 11
Project Name	GreetingCard2
Package Name	com.example.greetingcard
Programming Language	Kotlin
UI Toolkit	Jetpack Compose
Compile SDK Version	36
Target SDK Version	36
Minimum SDK Version	24
Emulator Used	Pixel 6 API 34

2. Project Creation and Execution Log

10:05 AM – Opened Android Studio.
10:07 AM – Selected "New Project".
10:08 AM – Chose "Empty Compose Activity" template.
10:09 AM – Named the project as **GreetingCard2**.
10:12 AM – Project created successfully.
11:13 PM – Gradle Sync started automatically.
11:30 PM – Emulator (Pixel 6 API 34) launched.
11:44 PM – Application successfully deployed and executed.

3. Project Structure Verification

The Project panel confirms the presence of essential Android files:

- MainActivity.kt
- AndroidManifest.xml
- res/values/strings.xml

This confirms correct project generation and Android application structure.

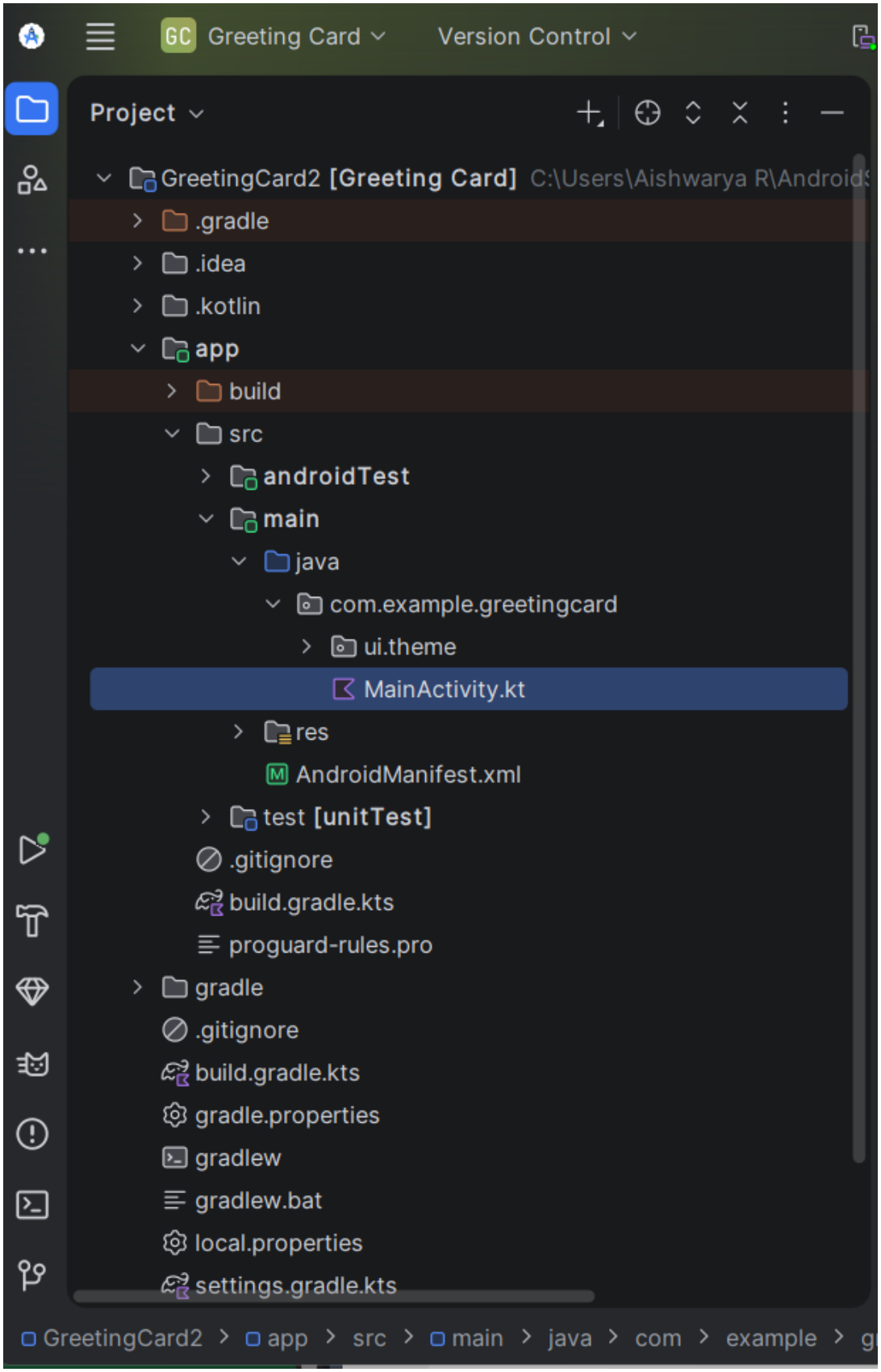

Figure 1: Project structure showing MainActivity.kt, AndroidManifest.xml, and resource files within the app module.

4. Compose Preview Verification

The Compose Preview panel displayed the UI component annotated with `@Preview`, confirming that the Compose rendering engine is functioning correctly.

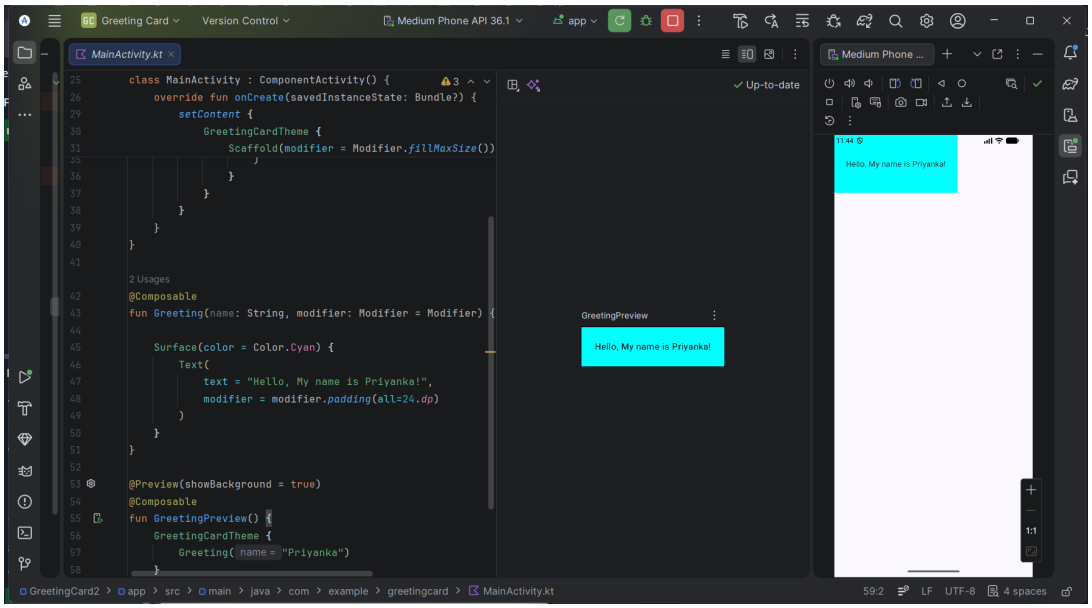

Figure 2: Compose Preview displaying the UI rendered using the @Preview annotation.

5. Running Application Output

The emulator successfully displayed the greeting message:

"Hello, My name is Priyanka!"

The UI utilized:

- Scaffold for layout structure
- Surface for background styling
- Text composable for displaying content
- Modifier.padding(24.dp) for spacing

Figure 3: Application successfully running on Pixel 6 API 34 emulator.

6. Explanation of Key Concepts

What does `setContent{}` do?

The `setContent{}` block defines the user interface of the activity using Jetpack Compose. Instead of XML layouts, Compose constructs the entire UI directly in Kotlin. This block acts as the entry point where composable functions are invoked to render the screen.

What does `@Composable` mean?

`@Composable` is an annotation that marks a function as a UI-building function in Jetpack Compose. These functions describe how UI components should appear and behave. Composable functions are reusable and can be combined to build complex interfaces.

7. Issues Encountered and Troubleshooting

During the setup process, the following issues were encountered:

- The Compose Preview was not displaying for two days.
- The Run button was disabled due to missing module configuration.
- The project was initially opened from the parent directory (AndroidStudioProjects), which prevented Gradle from detecting the app module.

Resolution:

- Closed Android Studio completely.
- Reopened only the correct project folder: GreetingCard2.
- Forced Gradle Sync.
- Created proper Run Configuration by selecting the **app** module.
- Connected the emulator and rebuilt the project.

After these steps, the Compose Preview appeared correctly and the application executed successfully.

8. First-Run Behaviour

During the first execution:

- Gradle downloaded required dependencies.
- SDK configuration was validated.
- The emulator booted successfully.
- The application compiled and deployed without errors.

9. Lessons Learned

- Opening the correct project root folder is essential for proper Gradle synchronization.
- Gradle Sync must complete successfully before running the application.
- Jetpack Compose eliminates XML layouts by using Kotlin-based UI definitions.
- Proper troubleshooting and documentation improve reproducibility.

10. Conclusion

The successful creation, troubleshooting, and execution of the GreetingCard2 project confirms that the Android Studio development environment is fully configured for Jetpack Compose development. The system is ready for further Compose-based UI experimentation and implementation.